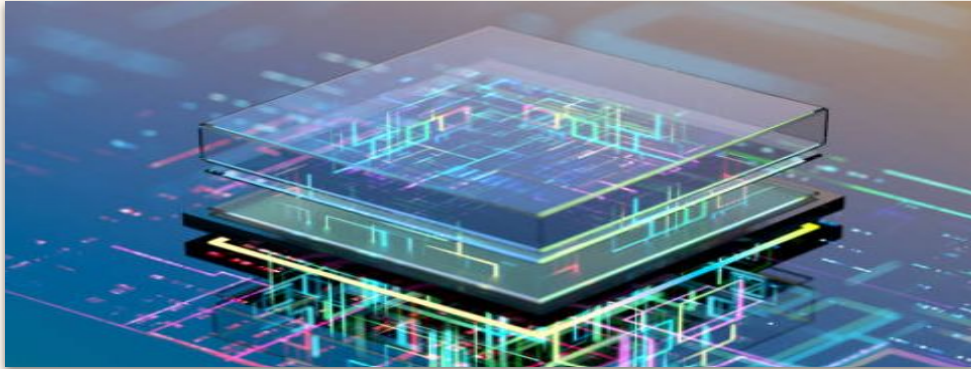

Mtech Software Engineering

Software Architecture - Assignment #2



Objective: To get familiar with the software architecture with real time applications

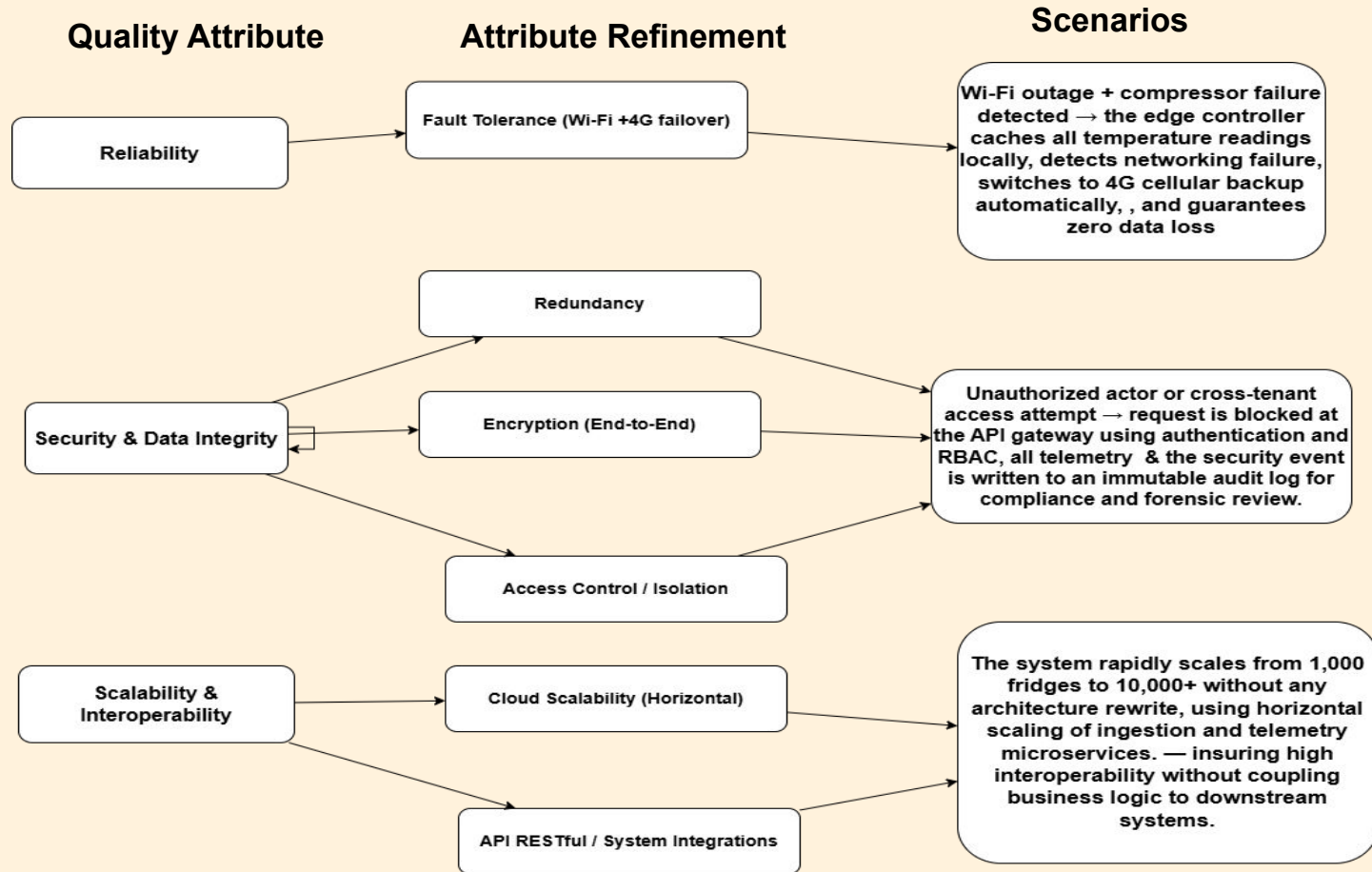
System Name: Medi-Fridge IoT Monitoring System

Salandri Shirisha - 2025TM93131 - 2025tm93131@wilp.bits-pilani.ac.in

Goals of the Medi-Fridge IoT Monitoring System

- To continuously monitor vaccine storage conditions and ensure safe, compliant, and reliable preservation of temperature-sensitive medical inventory.
- **Why this system is needed:** Protects vaccines from spoilage due to fridge failure Provides proof of compliance during medical audits
- **Architectural Drivers:** Must work even during network/power failures → requires edge failover + backup connectivity
- Must keep data secure and tamper-proof → requires encryption + immutable logging.

Utility tree of the top 3 ASRs for Medi-Fridge IoT Monitoring System



Architectural Significance

ASR 1 – Reliability (Fault Tolerance & Low-Latency Alerting)	Why it matters: Reliability is the heart of the Medi-Fridge VaccineShield System. The system must continue monitoring and alerting even during network or hardware failures.	Architectural Drivers: • Redundant • Connectivity:Edge-First Logic
ASR 2 – Security (Data Security & Privacy)	The system handles sensitive medical and compliance data. To meet healthcare regulations and prevent breaches, security must be built into every layer of the architecture—not added later.	• Hardware-Based Identity: • End-to-End Encryption & Access Control
ASR 3 – Scalability & Interoperability	As adoption grows across hospitals and pharmacies, the platform must easily handle thousands of devices and integrate with varied healthcare systems.	• Event-Driven & Serverless • DesignAdapter Pattern for Integration

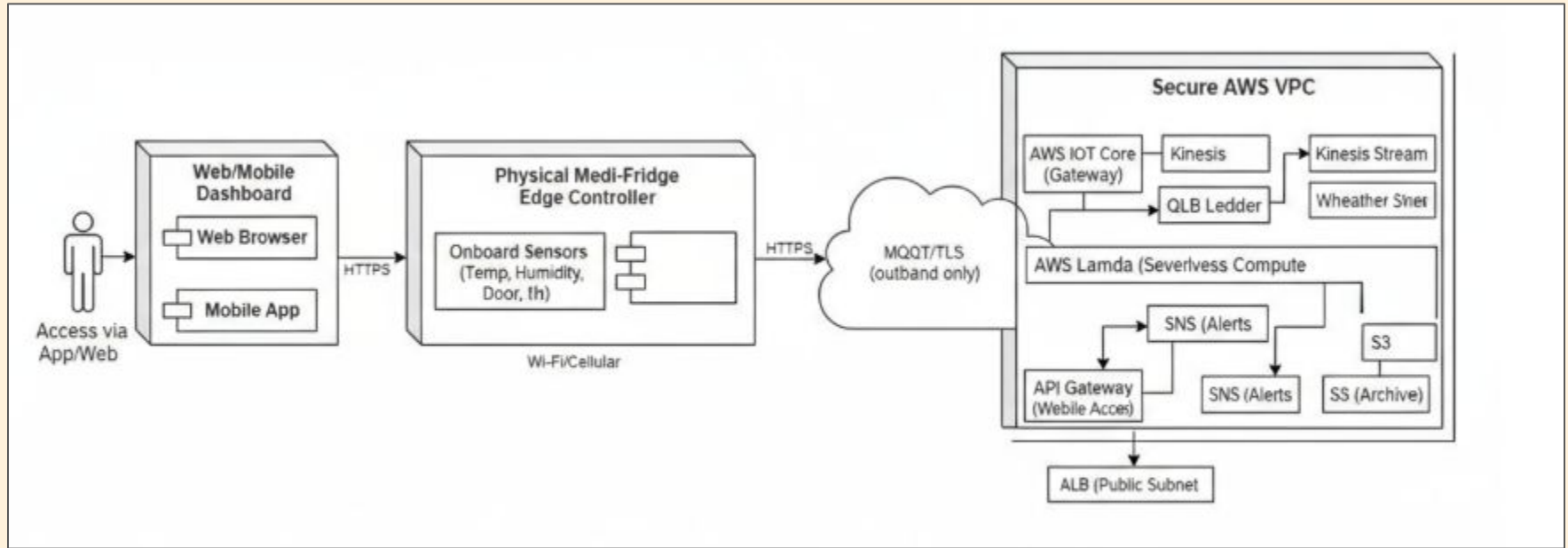
Tactics to achieve Top 3 ASRs

Quality Attribute	Scenario (Attribute Refinement)	Tactics
Scalability & Interoperability	System must handle thousands of fridges and integrate with external hospital / pharmacy systems	● Horizontal Auto-Scaling (Serverless/Containers): Handles sudden increases in device count/load. ● Decoupled Architecture using Message Streams: Ingest and processing layers scale independently. ● Adapter Pattern for Integrations: External PMS/EHR systems integrated via isolated microservices (FHIR/HL7).
Security	Unauthorized user/device attempts to access fridge data	● Hardware Identity (X.509 certificates): Each fridge is uniquely and cryptographically identified. ● mTLS End-to-End Encryption: All communication is encrypted in transit and data is encrypted at rest. ● Tenant-Based Access Control: API Gateway + authorizer checks tenant ID before granting access. ● Immutable Audit Logging

Tactics to achieve Top 3 ASRs (Cont..)

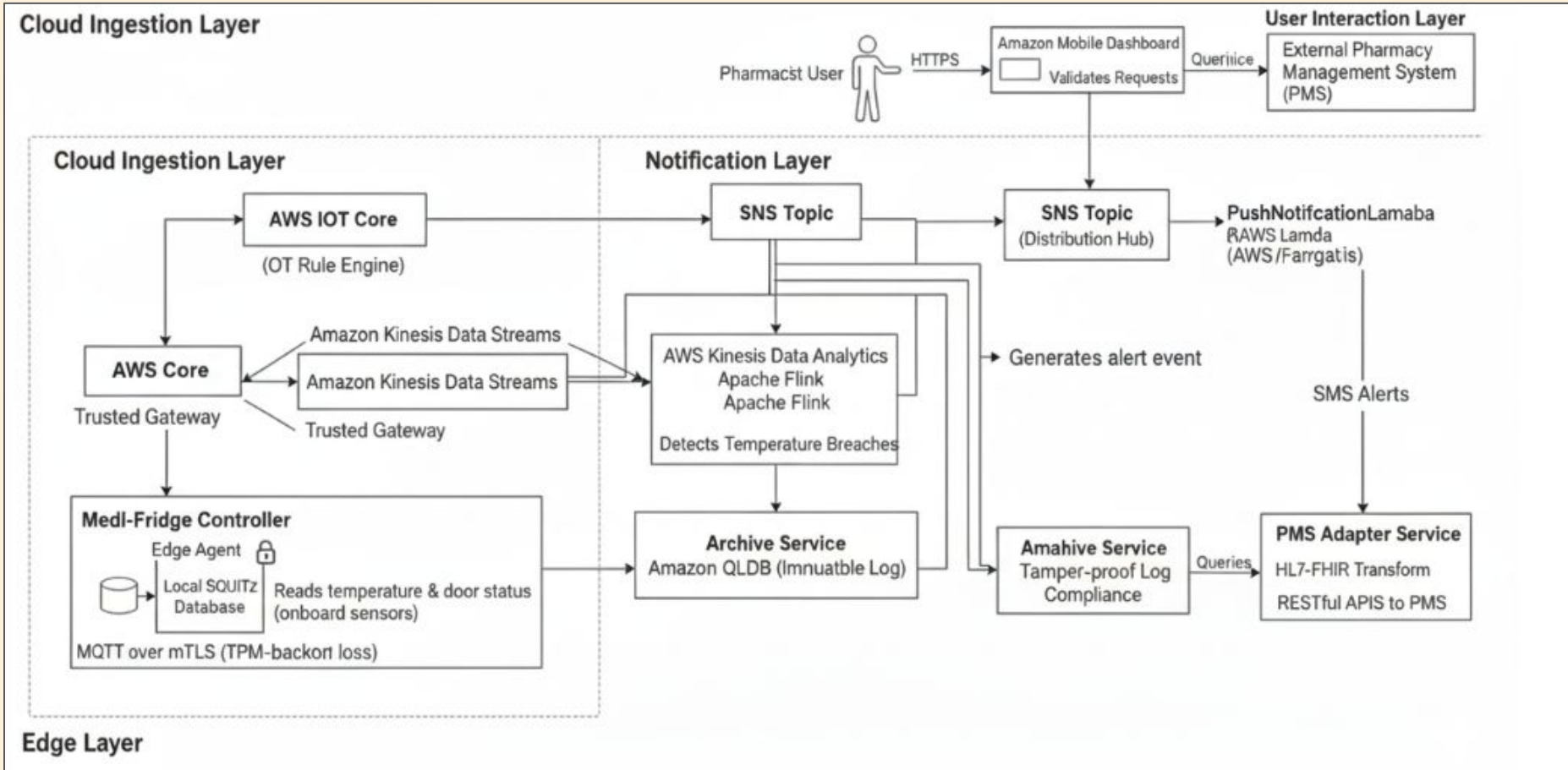
Quality Attribute	Scenario (Attribute Refinement)	Tactics
Reliability	Network failure + rising temperature the fridge must still send alerts and prevent data loss	• Edge Controller with Local Buffering: Device stores sensor readings locally during outages and retries when network is restored. • Redundant Connectivity (Wi-Fi + 4G): Automatic failover ensures alerts are still delivered. • Stream Processing Pipeline: Data is pushed to a real-time stream (Kinesis/Kafka) ensuring low-latency alerting. • CEP Rules Engine: Immediate detection of abnormal values and auto-triggering of alerts in <120 seconds.

Deployment Diagram



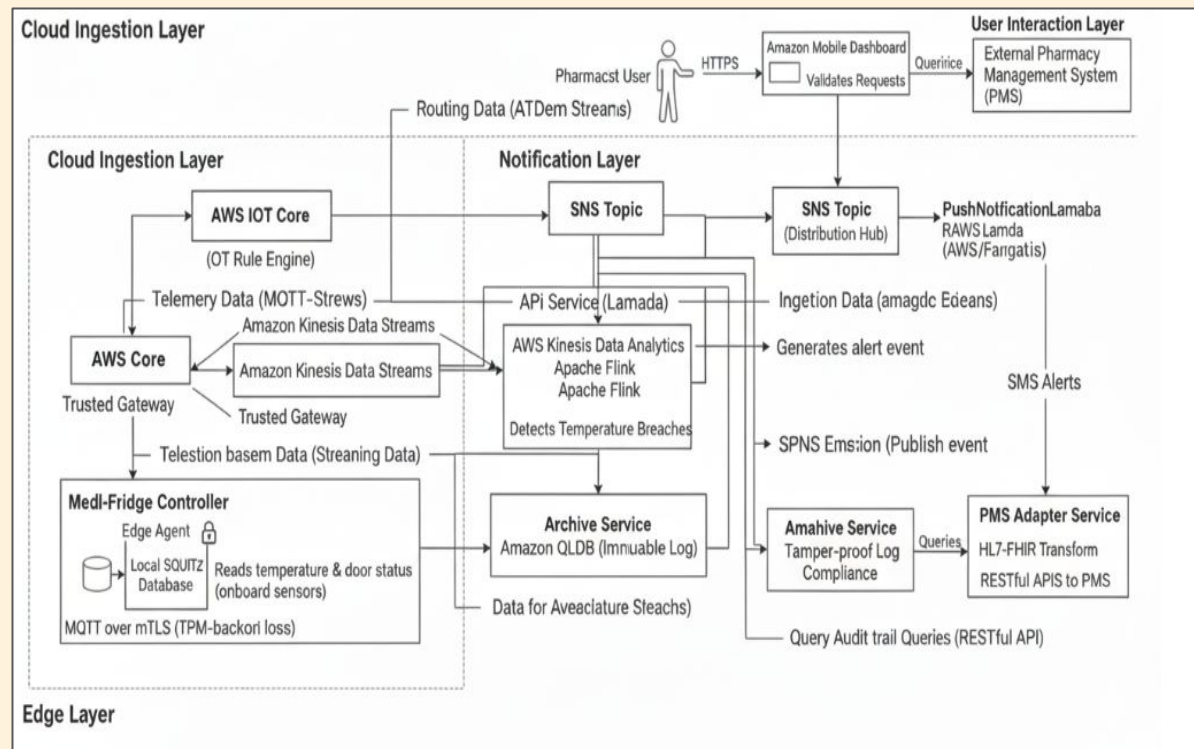
The deployment-the full flow of the **Medi-Fridge IoT System** from device to user. The **Edge Controller** collects sensor data and sends it securely via **MQTT/TLS** over Wi-Fi or cellular. In the **AWS Cloud**, data passes through **IoT Core** and **Kinesis** for real-time processing, stored in **QLDB** and **S3**, while **Lambda** and **SNS** handle alerts. Users access the system through a **secure web or mobile dashboard** via **API Gateway** and **ALB**, ensuring reliable, scalable, and secure monitoring.

Component & Connection View



Component & Connection View- Indicating the Communication Method

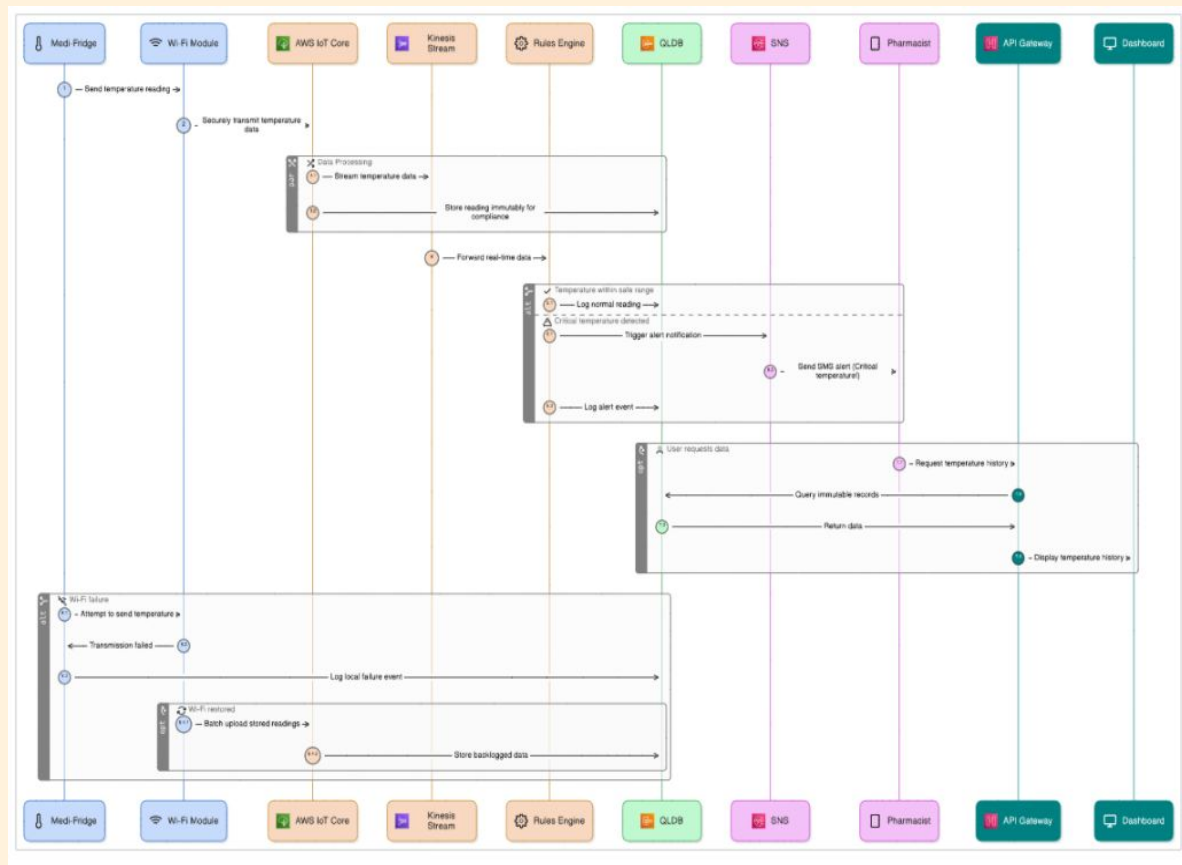
- ★ **Edge → Cloud (Data Upload)**
The Medi-Fridge Controller sends temperature and door-status data for secure, device-authenticated transfer.
- ★ **IoT Core → Streaming Layer (Data Routing)**
IoT Core forwards validated & enabling scalable, real-time data ingestion.
- ★ **Processing → Notification (Alert Generation)** detects temperature breaches and triggers an **SNS Topic**, broadcasting alerts to subscribers.
- ★ **User Interaction → Cloud (Query Access)** The pharmacist's app connects through **API Gateway** using **HTTPS REST API**.



Sequence Diagram – Critical Temperature Alert (Wi-Fi Failure)

The sequence shows how the Medi-Fridge remains reliable during a network failure. When a high temperature is detected, the fridge first tries Wi-Fi and then switches to 4G if it fails. It stores readings locally to prevent data loss and sends them securely once connected. The cloud instantly processes the data and sends an SMS alert to the pharmacist. **Automatic Failover:** Switches from Wi-Fi to 4G and buffers data locally. **Instant Alerts:** Cloud processing triggers SMS within seconds to warn the pharmacist.

Instant Alerts: Cloud processing triggers SMS within seconds to warn the pharmacist.



Architectural Pattern 1: Event-Driven Architecture (Stream Processing)

An Event-Driven Architecture (EDA) enables components to communicate asynchronously through events.

Producers emit events (e.g., “*new temperature reading*”), and consumers react independently. In VaccineShield, this is implemented as a real-time Stream Processing pipeline using **Amazon Kinesis Data Stream**.

Where it’s used: The cloud backend operates on an EDA model.

- **Event Producer:** AWS IoT Core sends *TempReading* events to Kinesis.
- **Event Consumers:**
 - Flink detects alerts.
 - Archive Service stores data in QLDB.
 - PMS Adapter sends data to external systems.
- **Secondary Event Bus:** Flink triggers *CriticalAlert* events to Amazon SNS for alerting.

Why it’s used (ASR links):

- **Reliability & Performance:** Real-time processing ensures sub-120s alert latency.
- **Scalability:** Kinesis decouples ingestion and processing, allowing independent scaling.
- **Interoperability:** New services can be added easily without changing existing components.

Architectural Pattern 2: Edge Computing

An architectural pattern that moves computation, data storage, and logic *out* of a centralized cloud and *onto* the device itself, at the "edge" of the network. This creates a "smart" device rather than a "dumb" sensor that just streams raw data.

- **Where it's used in VaccineShield:** The **Medi-Fridge Edge Controller** is the physical embodiment of this pattern.
 - **Local Logic:** It runs a "heartbeat" agent and the **failover logic** to automatically switch from Wi-Fi to the 4G cellular modem when a network failure is detected.
 - **Local Storage (Buffering):** It runs a **local SQLite database** to buffer all sensor readings during a network outage. This data is then securely flushed to the cloud when the connection is restored.
 - **Local Security:** It uses the onboard **TPM (Trusted Platform Module) chip** to securely store its private key and perform cryptographic operations (like signing data) *at the source*.
- **Why it's used (Links to ASRs):**
 - **Reliability (ASR #1):** This is the **only way** to achieve our primary ASR. We cannot trust the pharmacy's Wi-Fi. This pattern makes the system resilient to network failure and guarantees "0% data loss" by buffering locally.
 - **Performance (ASR #1):** A local alert (e.g., a loud buzzer on the fridge) can be triggered with zero-millisecond latency, even if the cloud is unreachable.

Key Learnings

★ ASRs Drive Every Major Design Choice

The biggest learning was understanding that **Architecturally Significant Requirements (ASRs)** are not just documentation they *define the system*. Functional goals like “log temperature” were simple; it was **Reliability** and **Security** that shaped the entire architecture. These ASRs ruled out basic designs and led to decisions like **edge computing**, **redundant connectivity**, and **immutable storage** for guaranteed accuracy and uptime.

★ Real Systems Combine Multiple Patterns

I learned that real-world architectures blend patterns rather than follow just one. The **Edge Computing** pattern ensured local reliability, while the **Event-Driven Architecture** provided scalability and modifiability in the cloud. Each layer used the right pattern for its problem, forming a single, cohesive design.

★ Architecture is About Managing Trade-offs

ASRs often conflict for example, strong **security** (TPMs, mTLS) adds overhead that can reduce **performance**. The real skill lies in balancing these trade-offs selecting efficient protocols like **MQTT** and hardware optimizations to maintain both security and low latency.

As an M.Tech student in Software Architecture, I learned how theoretical concepts like ASRs, architectural patterns, and quality attributes directly guide real system design decisions.

Happy Learning, thank you!